

Homework Assignment- LLM-Enhanced Honeypot and Attacker Interaction Analytics

This assignment builds on the deception and honeypot themes from the course, but requires students to design a fuller research-style exercise. Students must deploy or implement a controlled honeypot, create believable decoy artefacts, collect interaction logs, simulate attacker behaviour through playbooks in an isolated environment, and analyse whether LLM-generated or dynamically refreshed deception improves the quality of collected intelligence.

Learning Objectives

- Design a believable deception environment rather than a static dummy service.
- Use generative AI or carefully templated generation to create internally consistent decoy artefacts.
- Capture and structure interaction logs for later analysis.
- Analyse attacker playbooks, session depth, and likely intent from observed behaviour.
- Evaluate the realism, fingerprintability, and ethical boundaries of deception technology.

Provided Material

- No mandatory instructor material is provided.
- Students must create the honeypot environment, the decoy artefacts, and the simulated attacker interactions themselves.
- If a public honeypot framework, model API, or open-source attack simulation tool is used, it must be cited and clearly integrated into a reproducible workflow.

Operational / Ethical Constraints

- The honeypot must not be exposed to the public internet for this assignment unless there is an explicitly controlled, safe, and approved environment.
- Do not place real credentials, real personal data, or live secrets inside the decoy environment.
- The goal is to study defensive deception and attacker behaviour, not to attract or enable real-world harm.

Required Tasks

1. Design and Deploy the Honeypot Environment

Choose a constrained service profile and implement a believable environment for that service.

Steps:

- Select one primary interface to emulate, such as SSH, HTTP, or a simple internal file service.
- Define the threat model: what kind of attacker or playbook is the honeypot trying to attract or observe?
- Deploy the service in a controlled lab and document the components used, including any framework or custom code.
- Explain what makes the chosen environment believable and what its obvious limitations are.

Minimum evidence expected:

- A short architecture diagram or implementation description.
- A repeatable setup procedure.

2. Generate Believable Decoy Artefacts

Populate the environment with content that looks coherent enough to sustain attacker interest.

Steps:

- Create at least ten decoy artefacts spread across at least three categories, such as shell history, configuration files, fake credentials, service banners, logs, tickets, or administrator notes.
- At least five artefacts must be produced using an LLM or other generative workflow and not purely handwritten.
- Ensure internal consistency across usernames, hostnames, timestamps, software versions, and file references so that artefacts do not contradict one another.
- Describe the prompting or generation method used for the decoys and any manual curation that was necessary.
- State explicitly how you ensured that no real secrets or personal data were embedded.

Minimum evidence expected:

- A decoy inventory listing each artefact, its role in the story of the system, and whether it was LLM-generated or manually created.

3. Implement Logging and Session Capture

Collect enough structured information for later behavioural analysis rather than only storing raw text.

Steps:

- Log requests, commands, files accessed, timestamps, session identifiers, and any relevant source metadata.
- Store the logs in a structured format such as JSON or CSV in addition to any raw service logs.
- Define a minimal schema for the events so that sessions can be reconstructed and compared.
- Document what information is intentionally captured and what information is intentionally excluded for privacy or simplicity.

Minimum evidence expected:

- A sample of structured logs showing that complete sessions can be reconstructed.

4. Create Controlled Attacker Playbooks and Run Them

Simulate attacker behaviour in a repeatable way so the honeypot can be evaluated without waiting for real adversaries.

Steps:

- Define at least five playbooks, for example enumeration, credential search, configuration discovery, privilege-escalation probing, data-staging behaviour, or honeypot fingerprinting.
- Execute the playbooks only inside the isolated lab environment.
- Produce at least 15 sessions overall so that multiple behaviours can be compared.
- Vary the playbooks enough that some sessions are shallow and some are deeper or more goal-directed.

Minimum evidence expected:

- A short playbook catalogue with a one-sentence objective for each playbook.
- Evidence that the sessions were actually executed and logged.

5. Analyse Session Behaviour and Likely Intent

Turn the interaction logs into a behavioural analysis rather than leaving them as raw transcripts.

Steps:

- Compute at least three behavioural measures such as dwell time, number of artefacts touched, command diversity, request diversity, or path depth.
- Group or cluster sessions by tactic, intent, or behavioural similarity using either rule-based logic or an ML method.
- Provide at least three session summaries that explain what the simulated attacker appeared to be trying to achieve.
- Comment on whether the decoys caused deeper engagement than a minimal static environment would likely have caused.

Minimum evidence expected:

- A concise behavioural-analysis section backed by logs, tables, or plots.

6. Evaluate Deception Quality and Fingerprinting Risk

Assess whether the environment is convincing enough and identify how a sophisticated attacker might recognise it as a honeypot.

Steps:

- Identify at least three cues that could reveal the deception, such as inconsistent timestamps, unrealistic file content, repetitive banners, missing background noise, or unnatural command responses.
- Compare two decoy strategies, for example static content versus refreshed content, or manual content versus LLM-generated content.
- Discuss whether the LLM-generated artefacts improved realism, introduced hallucinations, or created detectable inconsistencies.
- Propose concrete improvements for the next iteration of the honeypot design.

Minimum evidence expected:

- A realism assessment that includes both strengths and weaknesses of the environment.

7. Discuss Ethics, Safety, and Operational Use

Conclude the exercise by addressing the policy implications of deception systems.

Steps:

- Discuss safe deployment boundaries, isolation, and data minimisation.
- Explain how the system would avoid harming legitimate users or collecting unnecessary information.
- Reflect on whether the additional realism created by an LLM changes the ethical profile of the honeypot.
- State what evidence would be needed before moving from a lab prototype to a production research deployment.

Minimum evidence expected:

- A short but thoughtful ethics and operational-readiness section in the report.

Deliverables

- The honeypot configuration or source code.
- The decoy artefacts or the generation pipeline used to create them.

- Structured session logs and a clear schema description.
- A report covering environment design, decoy generation, playbooks, behavioural analysis, realism assessment, and ethics.
- Screenshots or captured evidence showing the honeypot and the resulting interactions.
- A README and requirements file or equivalent run instructions.

Optional Advanced Tasks

- Regenerate selected decoy artefacts dynamically in response to attacker behaviour and measure whether session depth increases.
- Compare manual decoys and LLM-generated decoys using the same playbooks and analyse which approach yields richer attacker interaction.
- Train a small intent classifier or sequence model to predict whether a session is focused on reconnaissance, credential search, or privilege escalation.
- Use an LLM to summarise long sessions automatically, then compare the summaries against a manual analyst write-up.